WEEK 4 · LESSON 21 OF 24 · HANDOUT

# Feature Engineering and Pipelines

Study time: **about 55 minutes** (reading, video and activity)

▶ **Watch:** https://www.youtube.com/watch?v=C0_bh_5C5ro — open the lesson online to watch inside the course.

🎓 **Go deeper:** SciPy 2018 tutorial: Machine Learning with scikit-learn, Part 2 (scikit-learn core developers) — https://www.youtube.com/watch?v=gK43gtGh49o

**Your goal in this lesson:** Create better features from domain knowledge, and build a single scikit-learn Pipeline with a ColumnTransformer that imputes, encodes, scales and models — safely and repeatably.

## Better features beat fancier models

**Feature engineering** means creating new inputs that make the pattern easier to learn. It uses **domain knowledge** — what you know about the real world.

Any loan officer will tell you that the **size of a loan compared with income** matters more than either number alone. Let's check:

```
import numpy as np
import pandas as pd

rng = np.random.default_rng(11)
n = 2000
income = rng.lognormal(12.2, 0.45, n).round(-3)
loan = rng.uniform(20_000, 300_000, n).round(-3)
months = rng.integers(1, 61, n)
late = rng.poisson(0.6, n)
salary = rng.random(n) < 0.55
z = -3.2 + 2.5 * (loan / income) - 0.03 * months + 0.8 * late - 0.9 * salary
default = (rng.random(n) < 1 / (1 + np.exp(-z))).astype(int)
loans = pd.DataFrame({"income_rwf": income, "loan_rwf": loan, "months_customer": months,
                      "late_payments": late, "salaried": salary.astype(int),
                      "defaulted": default})
print(loans.shape, "default rate:", round(loans["defaulted"].mean(), 3))
loans.head()
```

```
loans["loan_to_income"] = loans["loan_rwf"] / loans["income_rwf"]
bands = pd.cut(loans["loan_to_income"], [0, 0.5, 1, 1.5, 10])
print(loans.groupby(bands, observed=True)["defaulted"].agg(["mean", "count"]).round(3))
```

Borrowers asking for less than half their monthly income default about **4%** of the time; those asking for more than 1.5 times their income default about **68%** of the time. One new column tells a very clear story.

```
from sklearn.model_selection import StratifiedKFold, cross_val_score
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

cv = StratifiedKFold(5, shuffle=True, random_state=42)
model = make_pipeline(StandardScaler(), LogisticRegression())
y = loans["defaulted"]
before = loans.drop(columns=["defaulted", "loan_to_income"])
after = loans.drop(columns=["defaulted"])
print("AUC before:", cross_val_score(model, before, y, cv=cv,
scoring="roc_auc").mean().round(3))
print("AUC after: ", cross_val_score(model, after, y, cv=cv,
scoring="roc_auc").mean().round(3))
```

AUC rises from about **0.866** to about **0.875** — a bigger gain than all the tuning in Lesson 17, from one sensible column.

## Common feature ideas

| Idea | Example |
|---|---|
| Ratios | loan ÷ income, price per m² |
| Dates | month, day of week, "is it a holiday?", days since last payment |
| Counts and totals | transactions in the last 30 days |
| Logs of skewed numbers | log(income) |
| Groups | district average rent (computed on training data only!) |
| Text | word counts, message length |

Every feature must be available **at prediction time**. "Number of reminders sent after the due date" would be leakage for predicting default at approval time.

## The problem pipelines solve

A real dataset mixes number and text columns, with missing values. Each needs different preparation, and every step must be learned from training data only — in every cross-validation fold. Doing this by hand is where most leakage bugs come from.

Let's make the homes data realistically messy:

```
import numpy as np
import pandas as pd

rng = np.random.default_rng(7)
n = 600
district = rng.choice(["Gasabo", "Kicukiro", "Nyarugenge"], n, p=[0.45, 0.35, 0.2])
bedrooms = rng.integers(1, 5, n)
size_m2 = (bedrooms * 28 + rng.normal(20, 12, n)).clip(25).round()
km_to_centre = rng.uniform(0.5, 15, n).round(1)
furnished = rng.random(n) < 0.3
base = {"Gasabo": 120_000, "Kicukiro": 90_000, "Nyarugenge": 100_000}
rent = (np.array([base[d] for d in district]) + 2_600 * size_m2 - 6_000 * km_to_centre
        + np.where(furnished, 60_000, 0) + rng.normal(0, 25_000, n)).round(-3)
homes = pd.DataFrame({"district": district, "bedrooms": bedrooms, "size_m2": size_m2,
                      "km_to_centre": km_to_centre, "furnished": furnished, "rent_rwf": rent})
print(homes.shape)
homes.head()
```

```
rng = np.random.default_rng(5)
messy = homes.copy()
messy.loc[rng.choice(len(messy), 30, replace=False), "size_m2"] = np.nan
messy.loc[rng.choice(len(messy), 15, replace=False), "district"] = np.nan
print(messy.isna().sum())
```

## ColumnTransformer + Pipeline

```
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_absolute_error

numbers = ["bedrooms", "size_m2", "km_to_centre"]
categories = ["district"]

prepare = ColumnTransformer([
    ("num", make_pipeline(SimpleImputer(strategy="median"), StandardScaler()), numbers),
    ("cat", make_pipeline(SimpleImputer(strategy="most_frequent"),
                          OneHotEncoder(handle_unknown="ignore")), categories),
    ("flag", "passthrough", ["furnished"]),
])
pipe = Pipeline([("prepare", prepare), ("model", LinearRegression())])

X = messy.drop(columns="rent_rwf")
y = messy["rent_rwf"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
pipe.fit(X_train, y_train)
print("Test MAE:", round(mean_absolute_error(y_test, pipe.predict(X_test))))
print(pipe.named_steps["prepare"].get_feature_names_out())
```

Read it from the inside out:

- **Number columns**: fill gaps with the median, then standardise.
- **District**: fill gaps with the most common district, then one-hot encode (unknown districts become zeros).
- **furnished**: passed through unchanged.
- Then the model.

`fit` learns every step from the training data; `predict` applies the same steps to new data. The test MAE is about **22,000 RWF** — close to the clean-data result, despite the gaps.

## One object does everything

```
new_homes = pd.DataFrame({
    "district": ["Gasabo", "Musanze"],        # Musanze never appeared in training
    "bedrooms": [2, 3],
    "size_m2": [80, np.nan],                   # missing size
    "km_to_centre": [3.0, 6.0],
    "furnished": [True, False],
})
print(pipe.predict(new_homes).round(-3))

scores = cross_val_score(pipe, X, y, cv=5, scoring="neg_mean_absolute_error")
print("Cross-validated MAE:", round(-scores.mean()))
```

Raw rows go in, predictions come out — missing values and unknown districts included. And `cross_val_score` re-fits the imputers, scaler and encoder inside every fold automatically: **no leakage, by design**. You can also put the pipeline straight into `GridSearchCV` and tune preparation choices (for example `prepare__num__simpleimputer__strategy`).

## Key terms

- **Feature engineering** — creating new, more useful inputs from existing data.
- **Domain knowledge** — understanding of the real-world problem.
- **Pipeline** — a chain of steps that is fitted and applied as one model.
- **ColumnTransformer** — applies different preparation to different columns.
- **SimpleImputer** — fills missing values (median, mean, most frequent).

**Remember this:** Good features come from understanding the problem; a Pipeline with a ColumnTransformer turns all preparation and the model into one leak-proof object.

Name: _____ Date: _____

## Activity

### Task 1

Add `log_income = np.log(income_rwf)` to the loans data. Does AUC improve further?

### Task 2

Swap `LinearRegression()` in the homes pipeline for `RandomForestRegressor(300, random_state=42)`. Which has the lower cross-validated MAE?

### Task 3

Add a `price_per_m2`-style feature to the homes data. Why would that be **leakage**?

### Task 4

Put the homes pipeline in `GridSearchCV` and compare median and mean imputation.

## Self-check

Think about your answers here, then take the quiz online — it is marked instantly and explains every answer.

1. Why did loan_to_income help the model?
   - A. It scales the data
   - B. It captures what really drives risk: the loan compared with income
   - C. It is a longer column
   - D. It removes missing values
2. Which feature would be leakage when predicting default at approval time?
   - A. Number of reminders sent after the due date
   - B. Months as a customer
   - C. Monthly income
   - D. Loan amount
3. What does ColumnTransformer do?
   - A. Draws charts
   - B. Trains several models
   - C. Deletes columns
   - D. Applies different preparation steps to different columns
4. What does SimpleImputer(strategy='median') do?
   - A. Fills missing values with the column's median
   - B. Deletes rows with gaps
   - C. Scales the data
   - D. Encodes text
5. Why is cross-validating a whole Pipeline leak-proof?
   - A. Pipelines cannot leak by magic
   - B. It uses the test set
   - C. All preparation is re-fitted on the training folds only
   - D. It skips preparation
6. A new home arrives with a missing size and a district never seen in training. What does the pipeline do?
   - A. Imputes the size and encodes the district as zeros, then predicts
   - B. Deletes the home
   - C. Asks the user
   - D. Crashes

© WAMUGII TECH SOLUTIONS · wamugii.kento.rw · Original course material — for enrolled students.